République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
École Supérieure Privée d'Ingénierie et de Technologies « ESPRIT »

DOCUMENT TECHNIQUE

Projet AWD · Validation 1

MeteoSphere

Plateforme météo intelligente en temps réel

Réalisé par	**Ines Boubakri, Mohamed Mehdi Hajjoun, Wala Bouraoui, Sinda Aouay, Mohamed Khalil Saidi, Motaz Nasfi**
Classe	**4DAS-I17**
Groupe	**Blossoms**
Encadrant	**[Nom de l'encadrant]**

Année Universitaire : 2026 / 2027

1. Description du sujet

La météo n'est plus une simple information : c'est un enjeu de sécurité. Les phénomènes extrêmes (canicules, inondations, tempêtes, feux de forêt) sont de plus en plus fréquents, tandis que les données disponibles (APIs, stations, signalements citoyens) restent dispersées, peu fiables localement et mal exploitées pour l'alerte.

MeteoSphere centralise les données météo en temps réel et les transforme en décisions utiles : une prévision, un niveau de risque, une alerte. La plateforme répond à deux problématiques principales :

- Comment centraliser et fiabiliser en temps réel des données météo venant de sources hétérogènes (APIs, capteurs, citoyens) ?
- Comment transformer ces données en niveaux de risque clairs et en alertes envoyées à temps aux bonnes personnes ?

La réponse s'articule en quatre étapes : collecter (APIs, stations, citoyens), prévoir (heure par heure, sur 7 jours), évaluer le risque (indice de 0 à 100 par zone) et alerter (email, SMS, push).

2. Les acteurs et les fonctionnalités

2.1 Les acteurs

Acteur	Type	Ce qu'il fait dans MeteoSphere
Citoyen	Humain	Consulte météo, prévisions et carte des risques ; s'abonne aux alertes ; signale la météo réelle.
Météorologue	Humain	Suit la précision des modèles ; gère les stations et saisit leurs mesures.
Administrateur	Humain	Gère utilisateurs et rôles, sources de données et seuils de risque.
APIs météo externes	Système	OpenWeatherMap et Open-Meteo fournissent les données, appelées par le service Ingestion.
Canaux de notification	Système	Email, SMS et push : reçoivent les alertes envoyées par le service Alertes.

2.2 Les principales fonctionnalités

Acteur	Fonctionnalités
Citoyen	Météo & prévisions · Carte des risques · Alertes personnalisées · Signaler la météo réelle
Météorologue	Précision des modèles · Gestion des stations · Saisie des mesures
Administrateur	Gestion des utilisateurs & rôles · Sources de données · Seuils de risque
Système	Collecte chaque minute · Recalcul automatique des modèles · Envoi des alertes

3. Le découpage en microservices

MeteoSphere est découpée en 7 microservices autonomes, chacun responsable d'un domaine métier précis et possédant sa propre base de données, auxquels s'ajoutent les briques d'infrastructure partagées (API Gateway, Eureka, Config Server, RabbitMQ, Redis).

Microservice	Port	Responsabilité
MS1 · Auth & Utilisateurs	8081	Authentification, émission du JWT, gestion des rôles et préférences.
MS2 · Ingestion météo	8082	Collecte météo toutes les minutes depuis les APIs externes.
MS3 · Prévisions météo	8083	Calcul des prévisions par ville, heure par heure et sur 7 jours.
MS4 · Alertes & Notifications	8084	Envoi des alertes personnalisées (email, SMS, push).
MS5 · Stations & Capteurs IoT	8085	Mesures des stations météo (température, humidité, vent, pression).
MS6 · Risques & Catastrophes	8086	Calcul de l'indice de risque (0–100) par zone géographique.
MS7 · Signalements citoyens	8087	Signalements citoyens en direct (photo, commentaires, votes).

Chaque microservice suit la même architecture interne en couches (Contrôleur → Service → Repository) : voir la section 7.2 « Patrons de conception ».

4. Les données gérées par chaque service

Chaque microservice choisit sa base de données selon la nature de ses propres données (règle « Database per Service »).

Microservice	Base	Principales données
Auth & Utilisateurs	PostgreSQL	users, roles, user_preferences, api_keys, refresh_tokens
Ingestion météo	MongoDB	raw_observations, providers, ingestion_jobs, ingestion_errors
Prévisions météo	PostgreSQL + Redis	cities, forecasts, forecast_models, forecast_accuracy, weather_conditions
Alertes & Notifications	MySQL	alert_rules, alerts, subscriptions, notification_channels, notification_logs
Stations & Capteurs IoT	MongoDB	stations, sensors, measurements, maintenance_logs
Risques & Catastrophes	PostgreSQL + PostGIS	risk_types, risk_zones (polygones), risk_thresholds, risk_assessments, risk_events
Signalements citoyens	MySQL	reports, report_photos, comments, votes, badges

Justification des choix de base de données

- PostgreSQL (Auth) : données très relationnelles, unicité de l'email et intégrité transactionnelle obligatoires.
- MongoDB (Ingestion) : schéma souple pour absorber des réponses JSON hétérogènes, fort volume d'écritures.
- PostgreSQL + Redis (Prévisions) : données structurées et liées, complétées d'un cache pour le service le plus consulté.
- MySQL (Alertes) : données relationnelles simples, journal transactionnel fiable pour tracer chaque envoi.
- MongoDB (Stations) : fort volume d'écritures en série temporelle, formats variables selon les capteurs.
- PostgreSQL + PostGIS (Risques) : requêtes géographiques (zones, polygones) et sortie GeoJSON pour la carte Leaflet.

- MySQL (Signalements) : modèle relationnel proche d'un réseau social, contrainte d'unicité « un vote par utilisateur ».

5. Les interactions entre les services

Les échanges entre microservices suivent deux modes : synchrone (appels REST via OpenFeign, pour les vérifications immédiates comme le JWT ou le profil utilisateur) et asynchrone (événements publiés sur RabbitMQ, pour propager une donnée sans coupler les services entre eux).

5.1 Auth & Utilisateurs

- Alertes et Signalements appellent Auth en synchrone (Feign) pour vérifier le token et récupérer le profil avant d'agir.
- Auth s'enregistre auprès d'Eureka au démarrage pour que les autres microservices puissent la trouver automatiquement.

5.2 Ingestion météo

- Un scheduler interroge OpenWeatherMap / Open-Meteo (HTTP) chaque minute pour récupérer les dernières mesures.
- Ingestion publie measurement.received sur RabbitMQ (async) : Prévisions et Risques l'écoutent pour recalculer leurs modèles.

5.3 Prévisions météo

- Ingestion publie measurement.received (async) : Prévisions recalcule ses modèles à chaque nouvelle mesure reçue.
- Prévisions publie forecast.updated (async) : Risques recalcule l'indice de risque dès qu'une nouvelle prévision est disponible.

5.4 Alertes & Notifications

- Risques publie risk.level.changed (async) : déclenche une alerte dès qu'un niveau change.
- Auth renvoie profil + canaux (sync) pour savoir qui et comment notifier.
- Alertes envoie ensuite l'email, le SMS ou la notification push correspondante.

5.5 Stations & Capteurs IoT

- Les capteurs (simulés) envoient leurs mesures en HTTP/MQTT aux Stations météo.
- Stations publie measurement.received (async) : Prévisions et Risques recalculent avec ces données terrain.

5.6 Risques & Catastrophes naturelles

- Ingestion et Stations publient measurement.received : Risques recalcule l'indice à chaque mesure.
- Signalements (optionnel) peut affiner l'indice avec les retours terrain (report.created).
- Risques publie risk.level.changed : Alertes déclenche les notifications correspondantes.

5.7 Signalements citoyens

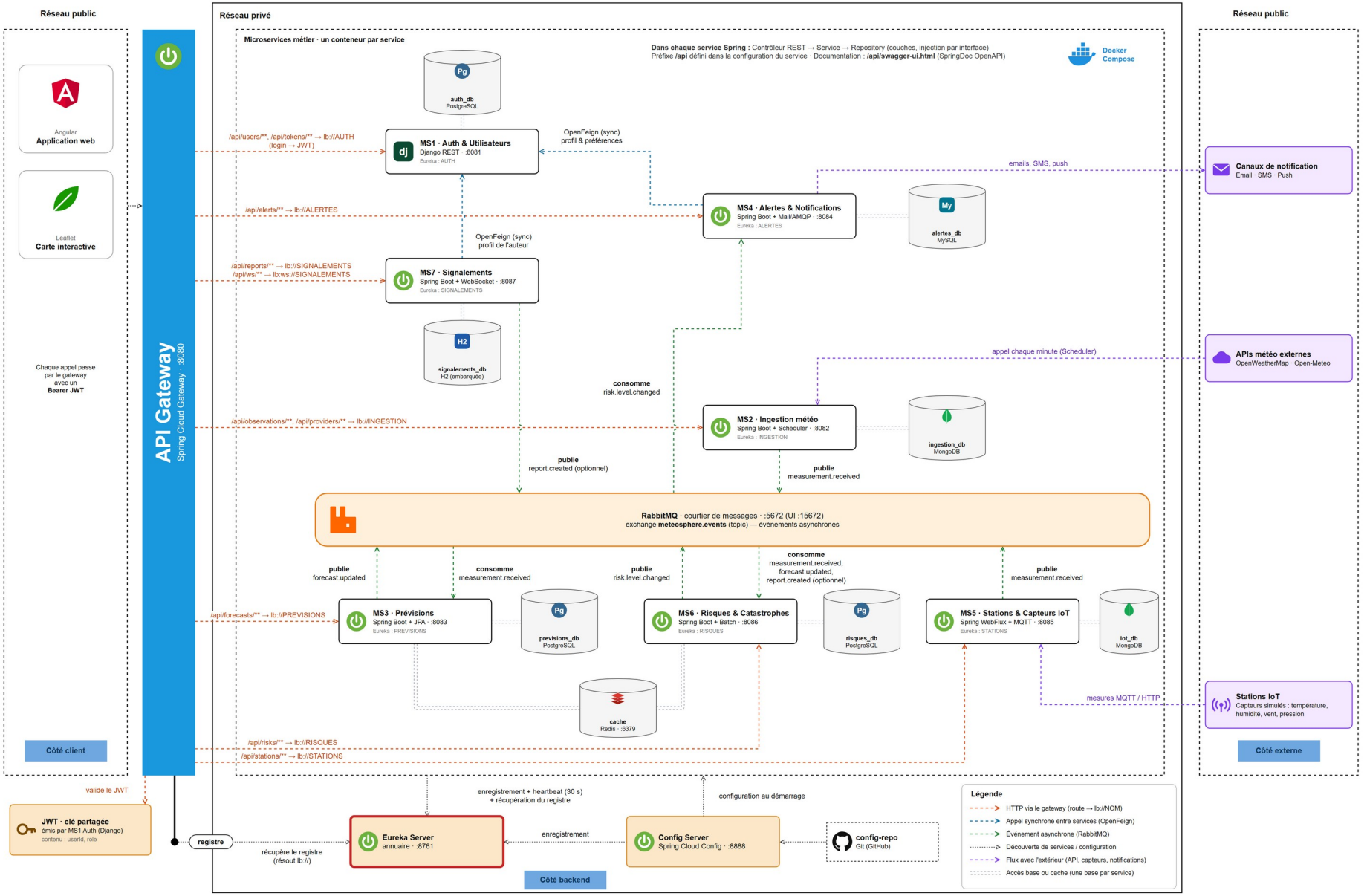
- Auth renvoie le profil de l'auteur (sync) pour afficher qui a signalé.
- Signalements peut publier report.created (optionnel) : Risques l'utilise pour affiner son indice.
- Les nouveaux signalements sont poussés en direct aux clients web via WebSocket (/topic/reports).

6. Le diagramme d'architecture globale

L'architecture globale repose sur un point d'entrée unique (API Gateway), un annuaire de services (Eureka) et une configuration centralisée (Config Server). Chaque microservice expose son API derrière la Gateway et communique avec les autres soit en synchrone (REST/Feign), soit en asynchrone via RabbitMQ. Redis sert de cache partagé pour les données consultées fréquemment (prévisions, carte des risques).

Suivre une mesure météo : architecture globale de la plateforme MeteoSphere

sources → Ingestion / IoT → RabbitMQ → Prévisions → Risques → Alertes → Notifications



Chaque service et son port

Composant	Port	Rôle
API Gateway	8080	Point d'entrée unique, vérifie le JWT
Eureka Server	8761	Annuaire des services (découverte)
Config Server	8888	Configuration centralisée au démarrage
MS1 · Auth & Utilisateurs	8081	Authentification, émission du JWT
MS2 · Ingestion météo	8082	Collecte météo toutes les minutes
MS3 · Prévisions météo	8083	Calcul des prévisions par ville
MS4 · Alertes & Notifications	8084	Envoi des alertes (email, SMS, push)
MS5 · Stations & Capteurs IoT	8085	Mesures des stations IoT
MS6 · Risques & Catastrophes	8086	Calcul de l'indice de risque
MS7 · Signalements citoyens	8087	Signalements citoyens en direct
RabbitMQ	5672 (UI 15672)	Messagerie asynchrone (événements)
Redis	6379	Cache partagé & données temps réel

7. Autres éléments nécessaires à la compréhension de la conception

7.1 Technologies choisies

Couche	Technologie	Pourquoi
Frontend	Angular + Leaflet	SPA réactive avec carte des risques interactive
Backend	Spring Boot 3 · Java 17	Standard microservices, écosystème Spring Cloud complet
Service commun	Django REST (Python)	Authentification partagée par l'équipe, émet le JWT
Gateway	Spring Cloud Gateway	Point d'entrée unique, routage, vérification du JWT
Découverte & config	Eureka + Config Server	Services trouvés par nom, configuration centralisée
Appels synchrones	OpenFeign (REST)	Réponse immédiate (ex. profil utilisateur)
Messagerie	RabbitMQ (exchange topic)	Événements asynchrones, services découplés
Bases de données	PostgreSQL · MySQL · MongoDB	Une base par service, adaptée à la nature des données
Cache & temps réel	Redis · WebSocket	Carte et prévisions rapides, fil de signalements en direct
Déploiement	Docker Compose	Toute la plateforme lancée en une seule commande

7.2 Patrons de conception

L'architecture est microservices entre les services (indépendants, chacun avec sa propre base), mais monolithique en couches à l'intérieur de chaque service (pas de découpage interne supplémentaire).

Patron	Niveau / Où	Pourquoi
Microservices	Architecture globale (système)	7 services autonomes, chacun avec sa propre base
Architecture en couches	À l'intérieur de chaque service	Contrôleur → Service → Repository, simple et lisible
API Gateway	Point d'entrée unique	Un seul accès, routage + vérification du JWT
Service Discovery	Entre les services	Les services se trouvent par leur nom (Eureka)
Database per Service	Chaque microservice	Pas de couplage par la donnée entre services
Repository Pattern	Couche d'accès aux données	Sépare la logique métier du stockage (Spring Data)
DTO	Échanges API	Isole le modèle interne du contrat exposé à l'API
Dependency Injection	Dans chaque couche Spring	Couplage faible, composants testables séparément
Event-Driven (pub/sub)	Entre services découplés	Réaction aux événements sans appel direct (RabbitMQ)

7.3 Répartition des tâches et gestion de projet

Un seul dépôt GitHub est utilisé, avec les tâches suivies dans GitHub Projects (Kanban : To Do / In Progress / Done). Chaque issue correspond à une tâche assignée à un membre, étiquetée par microservice.

Membre	Responsabilité principale
Wala Bouraoui	MS2 Ingestion · diagramme de cas d'utilisation général
Mohamed Mehdi Hajjoun	MS3 Prévisions · API Gateway + Config Server
Motaz Nasfi	MS4 Alertes · RabbitMQ + catalogue des événements
Mohamed Khalil Saidi	MS5 Stations · diagramme de classes global
Ines Boubakri	MS6 Risques · Eureka + docker-compose
Sinda Aouay	MS7 Signalements · README + squelette Angular
Toute l'équipe	MS1 Auth (Django) — chacun développe une partie

7.4 Planning prévisionnel

Organisation en 2 semaines de backend, 2 semaines de frontend puis 1 semaine d'intégration :

Période	Objectif
S8 (9 nov.)	Validation 1 — conception et projets créés
S9 (16 nov.)	Contrats OpenAPI, infrastructure, JWT Django décidé
S10 – S11 (23 nov.)	Backend métier — mocks, seeds, tests Postman
Vacances (7 déc.)	Frontend Angular — carte Leaflet, WebSocket
S12 (21 déc.)	Intégration, tests, docker-compose
S13 (finale)	Démo finale

7.5 Risques identifiés et parades

Risque	Parade
Dépendances entre services	Contrats OpenAPI figés en S5, mocks et faux événements RabbitMQ pour avancer en parallèle
JWT Django non validé par Spring	Algorithme, clé et contenu du token (userId, rôle) testés dès S5 entre Django et le gateway
Keycloak cité dans la grille	À valider avec l'enseignant ; Keycloak gardé en option si l'auth Django ne suffit pas
Quotas API / trop de conteneurs	2 fournisseurs météo + cache Redis ; profils docker-compose pour ne lancer que le nécessaire

7.6 Conclusion

- Découpage : 7 microservices autonomes, une base de données par service.
- Communication : REST synchrone (OpenFeign) et événements asynchrones (RabbitMQ).
- Organisation : dépôt Git unique et Kanban, développement backend → frontend → intégration.

Une architecture découplée, documentée et prête pour le développement.